



ÉCOLE NATIONALE SUPÉRIEURE D'INGÉNIEURS DE BRETAGNE SUD

Université Bretagne Sud

4^e année — Cybersécurité du logiciel

Rapport

Analyse et sécurisation de la plateforme

ShellOnYou

Authentification inter-services — Implicit Trust vs Zero Trust

Réalisé par

Abdelhakim HAFIDI

Aymeric ROBIN

Année universitaire 2025–2026

Table des matières

1	Introduction	3
1.1	Contexte du projet ShellOnYou	3
1.2	Périmètre de l'analyse	3
2	Présentation de l'architecture	4
2.1	Stack technique	4
2.2	Organisation des services	4
2.3	Réseau Docker appnet	4
3	Analyse des vulnérabilités	5
3.1	[Critique] Gateway sans token inter-service	5
3.2	[Critique] Gestion du SECRET_JWT et du secret inter-service	6
3.3	[Critique] ms-exercise : pas de vérification de l'appelant	8
3.4	[Critique] ms-other : même problème côté accessControl	8
3.5	[Modéré] PostgreSQL joignable par tous les services	8
3.6	[Déjà en place] Points positifs	8
4	Scénario d'attaque — Mouvement latéral	10
4.1	Conditions initiales	10
4.2	Étape 1 — Récupération des secrets	10
4.3	Étape 2 — Forge d'un JWT avec rôle Enseignant	10
4.4	Étape 3 — Appel direct à ms-exercise	11
4.5	Étape 4 — Accès direct à PostgreSQL	11
4.6	Impact potentiel	12
5	Implicit Trust vs Zero Trust	13
5.1	Définitions	13
5.1.1	Implicit Trust	13
5.1.2	Zero Trust	13
5.2	Ce que fait ShellOnYou aujourd'hui	13
5.3	Ce que Zero Trust changerait	13
5.4	Tableau comparatif	14

6	Corrections implémentées	15
6.1	Token inter-service et secret dédié	15
6.2	Build local de la gateway	16
7	Recommandations complémentaires	18
7.1	Segmentation réseau	18
7.2	Cloisonnement des secrets	18
7.3	Validation du token inter-service côté microservices	19
8	Conclusion	21

1. Introduction

1.1 Contexte du projet ShellOnYou

ShellOnYou est une plateforme web pédagogique pour initier les étudiants aux commandes Shell Unix. L'application repose sur une architecture microservices dockerisée, avec un front-end React/Redux, un back-end Node.js/TypeScript exposé via une API Gateway, et des bases PostgreSQL. La communication inter-services est censée suivre les principes du Zero Trust.

La montée en charge progressive de la plateforme — liée à l'ajout de fonctionnalités sociales comme les classements ou les remerciements entre pairs — rend les questions de sécurité et de disponibilité de plus en plus critiques.

1.2 Périmètre de l'analyse

Ce rapport porte sur l'axe sécurité du projet de 4^e année Cybersécurité du logiciel à l'ENSIBS. L'objectif est d'identifier les failles liées à la gestion de l'authentification et des autorisations, et de comparer concrètement les approches Implicit Trust et Zero Trust sur cette architecture.

La méthode retenue est une **analyse statique du code source** du dépôt SOY_MS_HTTP, disponible sur l'organisation GitHub ShellOnYou : https://github.com/ShellOnYou/SOY_MS_HTTP. Les fichiers examinés sont les suivants :

- `docker-compose.yml` — configuration réseau et services,
- `variables.env` — secrets et variables d'environnement,
- `gateway-http/index.js` — logique de proxy,
- `other-http/index.js` et `middlewares/accessControl.js`,
- `exercise-http/routes/exerciseProduction.js` et `middlewares/accessControl.js`.

Les vulnérabilités ont été détectées en croisant la topologie réseau Docker avec les middlewares d'authentification présents dans chaque service.

2. Présentation de l'architecture

2.1 Stack technique

- **Front-end** : React / Redux
- **Back-end** : Node.js / TypeScript
- **Base de données** : PostgreSQL
- **Orchestration** : Docker Compose

2.2 Organisation des services

Le dépôt SOY_MS_HTTP est structuré autour de quatre composants :

- **gateway-http** : point d'entrée unique, proxifie les requêtes du front vers les microservices,
- **exercise-http** (**ms-exercise**) : gestion des exercices et de leur mise en production,
- **other-http** (**ms-other**) : gestion des utilisateurs, routes admin et exercices complémentaires,
- **soy-db** : instance PostgreSQL hébergeant l'ensemble des données.

2.3 Réseau Docker appnet

Tous les services partagent un seul réseau bridge **appnet**, sans aucune segmentation :

Listing 2.1 – docker-compose.yml — réseau unique partagé

```
1 networks:
2   appnet:
3     driver: bridge
4 # postgres, ms-exercise, ms-other, gateway
5 # -> tous joignables entre eux, sans restriction
```

C'est ce point de départ qui rend possible l'ensemble des scénarios d'attaque décrits dans ce rapport.

3. Analyse des vulnérabilités

3.1 [Critique] Gateway sans token inter-service

La gateway se contente de transmettre les requêtes telles quelles aux microservices, sans prouver sa propre identité. Tout container sur `appnet` peut donc appeler `ms-exercise` ou `ms-other` directement, sans jamais passer par la gateway.

Listing 3.1 – gateway-http/index.js — proxy sans auth

```
1 app.use("/api/exercise-production", (req, res) => {
2   apiProxy.web(req, res, {
3     target: 'http://ms-exercise:' + process.env.PORT
4       + "/api/exercise-production"
5   })
6   // <- aucun header d'authentification inter-service
7 });
```

Schéma de jetons recommandé. Dans une architecture Zero Trust, deux jetons distincts coexistent, chacun connu d'un périmètre différent :

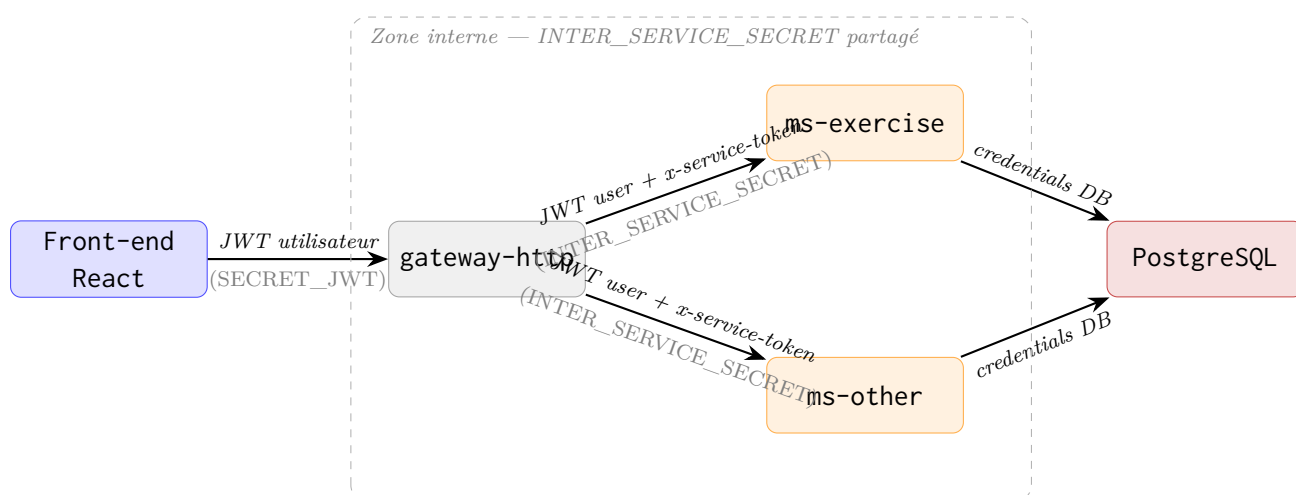


FIGURE 3.1 – Circulation des jetons dans l'architecture Zero Trust cible. `SECRET_JWT` est connu du front-end et de la gateway uniquement. `INTER_SERVICE_SECRET` est partagé entre la gateway et les microservices (mais *pas* le front-end). PostgreSQL ne connaît ni l'un ni l'autre.

Avec cette organisation, deux types de jetons sont nécessaires :

- **JWT utilisateur** (signé avec `SECRET_JWT`) : prouve l'identité de l'utilisateur final. Connu de la gateway, qui le valide à l'entrée, et des microservices, qui l'utilisent pour les contrôles de rôle.
- **Token inter-service** (signé avec `INTER_SERVICE_SECRET`) : prouve que la requête émane bien de la gateway et non d'un appelant arbitraire sur `appnet`. Inconnu du front-end.

La question de partager un même secret entre deux microservices qui se parleraient directement est légitime : dans l'architecture actuelle (v5), seule la gateway appelle les microservices, donc un unique `INTER_SERVICE_SECRET` suffit. Si deux microservices devaient se parler, il faudrait soit un secret partagé supplémentaire, soit un mécanisme d'identité par certificat (mTLS), mais cela dépasse le périmètre de la v5.

3.2 [Critique] Gestion du `SECRET_JWT` et du secret inter-service

Situation réelle dans le dépôt

Une lecture rapide pourrait laisser penser que `variables.env` est monté directement dans chaque container. Ce n'est pas le cas : ce fichier réside à la racine du projet, au même niveau que `docker-compose.yml`, et Docker Compose l'utilise via la directive `env_file` ou la substitution de variables dans le compose. Il n'est donc pas physiquement injecté dans le système de fichiers des containers.

En revanche, les variables qu'il contient *sont* injectées dans l'environnement de chaque container au démarrage. Un attaquant disposant d'un shell dans n'importe quel container peut les lire via la commande Unix `env` :

Listing 3.2 – Lecture des variables d'environnement depuis un container compromis

```
1 $ env | grep -E 'JWT|POSTGRES'
2 SECRET_JWT=your_custom_jwt_secret
3 POSTGRES_PASSWORD=anonymous
```

Vraie problématique : un secret commun à tous les services

Le problème n'est pas que le secret soit stocké en clair — il faut bien qu'il soit injecté d'une façon ou d'une autre dans les containers pour être utilisable. La vraie faille est double :

1. **Tous les containers reçoivent toutes les variables**, y compris celles dont ils n'ont pas besoin. Le front-end n'a pas à connaître `POSTGRES_PASSWORD`, et PostgreSQL n'a pas à connaître `SECRET_JWT`.

2. **SECRET_JWT est identique pour tous les services.** Quiconque compromet un container — même celui qui n'a aucune raison de signer des JWT — obtient le secret et peut forger des tokens valides pour l'ensemble de la plateforme.

Peut-on vraiment protéger un secret contre un attaquant ayant un shell ?

La question mérite d'être posée honnêtement : *comment et où stocker le secret autrement, si de toute façon il finit dans les variables d'environnement du container ?*

La réponse courte est que **non** — aucun mécanisme ne protège totalement un secret contre un attaquant qui dispose déjà d'un shell dans le container. Les variables d'environnement sont lisibles via `env`, Docker Secrets injecte les secrets sous forme de fichiers dans `/run/secrets/` (moins exposé que `env`, mais lisible par un shell), et même un gestionnaire comme HashiCorp Vault finit par injecter un token ou une valeur dans le processus, récupérable par un attaquant suffisamment avancé.

Ce constat change la façon d'aborder le problème : **la vraie défense ne consiste pas à rendre le secret illisible, mais à limiter ce qu'un attaquant peut faire avec s'il le vole.** C'est précisément le rôle du token inter-service à durée de vie courte (30 secondes) : même si `INTER_SERVICE_SECRET` est récupéré, la fenêtre d'exploitation est très étroite — l'attaquant doit forger et utiliser un token dans la même demi-minute, ce qui complique significativement une attaque automatisée.

Les améliorations concrètes restent néanmoins utiles, par ordre croissant de protection :

1. **Cloisonnement par `env_file`** : chaque service ne reçoit que les variables qu'il consomme réellement. Compromis sur un container qui n'a pas `SECRET_JWT` → impossible de forger un JWT utilisateur. C'est simple à mettre en place et déjà efficace.
2. **Docker Secrets** (mode Swarm) : le secret est injecté comme fichier dans `/run/secrets/`, non visible dans `env`. Réduit l'exposition aux outils qui lisent naïvement les variables d'environnement, mais n'empêche pas un attaquant avec un shell de lire le fichier.
3. **HashiCorp Vault / équivalent** : injection dynamique à la demande, secrets à usage unique, audit des accès, révocation immédiate. Le meilleur niveau de contrôle, mais aussi le plus complexe à opérer.

Dans le contexte de la v5, le cloisonnement par `env_file` couplé à des tokens inter-services à courte durée de vie est un compromis réaliste et déjà très supérieur à la situation actuelle.

3.3 [Critique] ms-exercise : pas de vérification de l'appelant

`auth.isAuth` vérifie que le JWT utilisateur est valide, mais ne contrôle pas du tout que la requête provient de la gateway. Un appel direct avec un JWT forgé passe sans problème.

Listing 3.3 – `exercise-http/routes/exerciseProduction.js`

```
1 router.post("/api/exercise-production",
2   auth.isAuth, connection,
3   // verifie le token user, mais pas qui appelle
4   function (req, res) {
5     ControllerExerciseProductionAPI.create(req, res);
6   });
```

3.4 [Critique] ms-other : même problème côté `accessControl`

`accessControl.checkConnection` délègue entièrement à `auth.isAuth`. Résultat identique : l'origine de la requête n'est jamais contrôlée.

Listing 3.4 – `other-http/middlewares/accessControl.js`

```
1 checkConnection: async function(req, res, next) {
2   auth.isAuth(req, res, next)
3   // verification utilisateur uniquement
4   // aucun controle sur l'identite de l'appelant
5 }
```

3.5 [Modéré] PostgreSQL joignable par tous les services

PostgreSQL est sur `appnet` au même titre que les autres containers. Un service compromis peut s'y connecter directement avec les credentials récupérés dans les variables d'environnement.

Listing 3.5 – `other-http/index.js` — connexion directe

```
1 const pool = new pg.Pool({
2   host: process.env.POSTGRES_HOST, // "postgres"
3   password: process.env.POSTGRES_PASSWORD, // "anonymous"
4 });
```

3.6 [Déjà en place] Points positifs

— JWT utilisateur vérifié à l'entrée de la gateway,

- CORS restreint à l'URL du front-end,
- `security_opt: no-new-privileges` sur tous les containers.

4. Scénario d’attaque — Mouvement latéral

Un **mouvement latéral** (*lateral movement*) désigne la capacité d’un attaquant, une fois qu’il a pris pied sur un premier système d’un réseau, à se déplacer vers d’autres systèmes ou ressources de ce même réseau sans avoir à forcer une nouvelle frontière périmétrique [1]. C’est précisément ce que permet l’architecture actuelle de ShellOnYou : compromettre un seul container ouvre l’accès à l’intégralité du réseau interne **appnet**.

4.1 Conditions initiales

L’attaquant a réussi à compromettre un container présent sur **appnet** — par exemple via une vulnérabilité dans une dépendance npm ou une mauvaise gestion d’une entrée utilisateur. Il dispose d’un accès shell sur ce container et peut joindre tous les autres services du réseau interne.

4.2 Étape 1 — Récupération des secrets

Depuis le container compromis, la lecture des variables d’environnement est immédiate et ne nécessite aucun privilège particulier :

Listing 4.1 – Lecture des secrets depuis le container

```
1 $ env
2 SECRET_JWT=your_custom_jwt_secret
3 POSTGRES_PASSWORD=anonymous
4 POSTGRES_HOST=postgres
```

L’attaquant dispose maintenant du secret de signature JWT et du mot de passe de la base de données.

4.3 Étape 2 — Forge d’un JWT avec rôle Enseignant

Avec `SECRET_JWT`, il génère un token valide côté serveur, impossible à distinguer d’un token légitime :

Listing 4.2 – Forge d’un JWT privilégié

```
1 const jwt = require('jsonwebtoken');
2
3 const token = jwt.sign(
4   {
5     id: 42,
6     email: 'attaquant@evil.com',
7     role: 'Enseignant'
8   },
9   'your_custom_jwt_secret',
10  { expiresIn: '2h' }
11 );
12
13 console.log(token);
14 // eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

4.4 Étape 3 — Appel direct à ms-exercise

Il contacte ms-exercise directement depuis appnet, sans passer par la gateway ni subir les contrôles CORS :

Listing 4.3 – Bypass de la gateway via appel direct

```
1 $ curl -X POST http://ms-exercise:5001/api/exercise-production \
2   -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
3     ..." \
4   -H "Content-Type: application/json" \
5   -d '{"title":"exercice-piege","content":"rm -rf /"}'
6 HTTP/1.1 200 OK
7 {"message":"Exercise created","id":99}
```

ms-exercise accepte la requête car auth.isAuth valide le JWT — il n’a aucun moyen de savoir que l’appelant n’est pas la gateway.

4.5 Étape 4 — Accès direct à PostgreSQL

En parallèle, l’attaquant se connecte directement à la base :

Listing 4.4 – Connexion directe à PostgreSQL

```
1 $ psql -h postgres -U postgres -W
2 Password: anonymous
3
```

```
4 postgres=# \dt
5           Liste des relations
6 Schema |      Name      | Type  | Owner
7 -----+-----+-----+-----
8 public | users          | table | postgres
9 public | exercises      | table | postgres
10 public | sessions       | table | postgres
11
12 postgres=# SELECT email, password_hash, role FROM users;
13 -- dump complet de tous les comptes utilisateurs
```

4.6 Impact potentiel

- Exfiltration de l'ensemble des données utilisateurs (emails, hashes de mots de passe),
- Création ou modification d'exercices avec contenu malveillant,
- Élévation de privilège vers le rôle **Admin** par modification directe en base,
- Persistance via backdoor insérée dans la base de données.

5. Implicit Trust vs Zero Trust

5.1 Définitions

5.1.1 Implicit Trust

L’Implicit Trust part du principe que tout service à l’intérieur du réseau interne est légitime et digne de confiance. La frontière réseau (ici **appnet**) constitue l’unique périmètre de sécurité. Une fois à l’intérieur, les services communiquent librement sans s’authentifier mutuellement.

5.1.2 Zero Trust

Le Zero Trust inverse ce postulat : « *ne jamais faire confiance, toujours vérifier* ». Chaque service doit prouver son identité à chaque appel, quelle que soit sa position dans le réseau. Le réseau interne est traité avec le même niveau de méfiance que l’Internet public.

5.2 Ce que fait ShellOnYou aujourd’hui

L’architecture actuelle est un cas d’école d’Implicit Trust :

- La gateway ne s’authentifie pas auprès des microservices,
- Les microservices font confiance à toute requête arrivant sur **appnet** dès lors qu’elle porte un JWT utilisateur valide,
- PostgreSQL accepte n’importe quelle connexion venant du réseau interne.

La compromission d’un seul container suffit à tout compromettre, comme le montre le scénario du chapitre précédent.

5.3 Ce que Zero Trust changerait

Les mesures suivantes s’appliquent à la v5, périmètre de notre analyse :

- **Token inter-service** : la gateway signe chaque requête sortante avec `INTER_SERVICE_SECRET`. Les microservices rejettent toute requête ne portant pas ce token valide dans `x-service-token`,

- **Segmentation réseau** : PostgreSQL est déplacé sur un sous-réseau **dbnet** inaccessible à la gateway et au front,
- **Secrets cloisonnés** : chaque service ne reçoit que les variables dont il a besoin, jamais un fichier global partagé,
- **Tokens à courte durée de vie** : expiration à 30 secondes pour limiter la fenêtre d'exploitation en cas de vol.

Perspective v7. La version 7 de ShellOnYou va plus loin dans cette direction en décomposant l'architecture en microservices plus fins (**ms-user**, **ms-skill**, **ms-exercise-student**, **ms-exercise-teacher**) chacun doté de sa propre base PostgreSQL dédiée. Ce cloisonnement des données réduit mécaniquement la surface d'un mouvement latéral : compromettre **ms-skill** ne donne plus accès aux tables utilisateurs, qui résident dans une base séparée pilotée par **ms-user**. C'est une illustration concrète du principe Zero Trust appliqué au stockage, au-delà du seul plan réseau.

5.4 Tableau comparatif

Critère	Implicit Trust (actuel)	Zero Trust (cible)
Vérification de l'origine	✗ Aucune	✓ Token inter-service
Segmentation réseau	✗ Flat network	✓ Sous-réseaux isolés
Accès PostgreSQL	✗ Tous les services	✓ Services autorisés uniquement
Gestion des secrets	✗ Fichier global	✓ Par service
Surface d'attaque	✗ Totale	✓ Limitée par périmètre
Mouvement latéral	✗ Trivial	✓ Bloqué par défaut

TABLE 5.1 – Implicit Trust vs Zero Trust — ShellOnYou

6. Corrections implémentées

6.1 Token inter-service et secret dédié

Ces deux points sont liés et constituent une seule et même correction : introduire un `INTER_SERVICE_SECRET` distinct de `SECRET_JWT`, et faire en sorte que la gateway l'utilise pour signer un token ajouté à chaque requête sortante.

Ajout du secret dans `variables.env`

Listing 6.1 – `variables.env` — secret inter-service ajouté

```
1 SECRET_JWT=your_custom_jwt_secret
2 INTER_SERVICE_SECRET=your_inter_service_secret
3 POSTGRES_PASSWORD=anonymous
```

Génération et injection du token côté gateway

Listing 6.2 – `gateway-http/index.js` — token inter-service

```
1 const interToken = jwt.sign(
2   { iss: 'gateway', aud: 'internal-service' },
3   process.env.INTER_SERVICE_SECRET,
4   { expiresIn: '30s' }
5 );
6
7 app.use("/api/exercise-production", (req, res) => {
8   req.headers['x-service-token'] = interToken;
9   apiProxy.web(req, res, {
10     target: 'http://ms-exercise:' + process.env.PORT
11       + "/api/exercise-production"
12   });
13 });
```

Le token généré est vérifiable sur `jwt.io` : claims `iss: gateway`, `aud: internal-service`, expiration 30 secondes.

Schéma de communication après correction

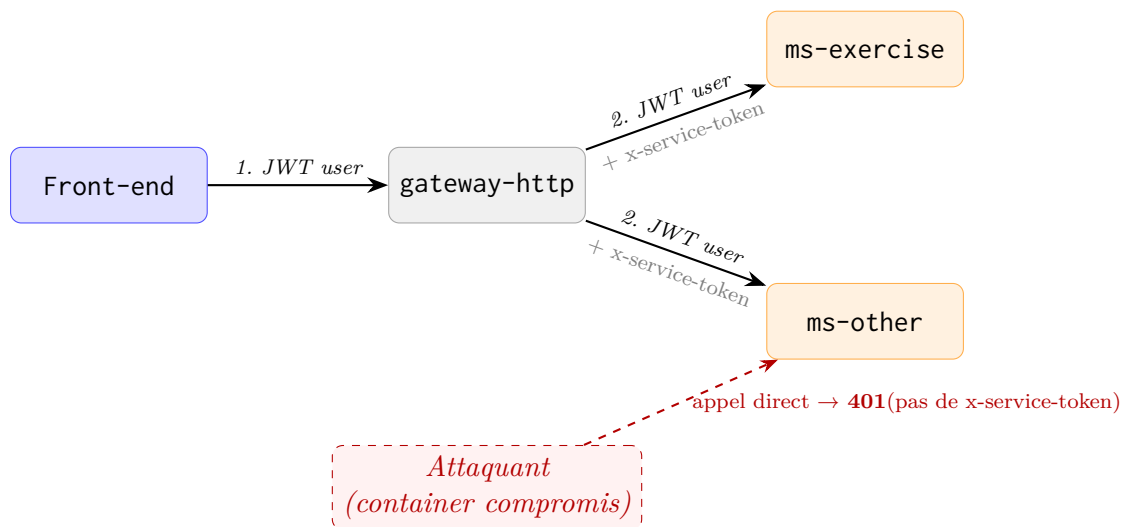


FIGURE 6.1 – Après correction : un appel direct depuis un container compromis est rejeté par `ms-exercise` car il ne porte pas de `x-service-token` valide. Seule la gateway, qui connaît `INTER_SERVICE_SECRET`, peut produire ce token.

6.2 Build local de la gateway

Le `docker-compose.yml` du dépôt `SOY_MS_HTTP` est conçu pour un déploiement *client* : toutes les images sont tirées depuis le registry `icws24submission` via la directive `image:`, et non construites localement. C’est un choix délibéré — un utilisateur souhaitant juste déployer la plateforme n’a pas à cloner le code source ni à le compiler.

Il est important de noter que l’image publiée sur ce registry *est elle-même construite depuis ce même code source*. Passer par le registry ou builder localement revient donc au même en termes de confiance dans le code exécuté — l’argument de sécurité (« construire soi-même pour savoir ce qu’on fait tourner ») ne s’applique pas vraiment ici puisque les deux chemins partent du même dépôt.

La **seule raison** pour laquelle nous devons builder localement la gateway est pratique : nous avons modifié son code source pour y intégrer la génération du token inter-service. L’image publiée sur le registry ne contient évidemment pas ces modifications, donc tirer cette image écraserait notre correction :

Listing 6.3 – `docker-compose.yml` — build local de la gateway

```

1 gateway:
2   build:
3     context: ./gateway-http
4     dockerfile: Dockerfile
5   # au lieu de : image: icws24submission/gateway:latest
  
```

```
6 # Raison : le code de la gateway a ete modifie localement ;  
7 # l'image du registry ne contient pas le token inter-service.
```

Cette modification ne concerne *que* la gateway, car c'est le seul service dont nous avons effectivement changé le code. Les autres containers (`ms-exercise`, `ms-other`, `soy-db`) continuent d'être tirés depuis le registry : nos corrections pour ces services restent des recommandations (middleware de validation côté microservices, segmentation réseau) qui n'ont pas encore été implémentées dans leur code source — elles sont décrites au chapitre suivant. Si ces corrections étaient intégrées, il faudrait alors appliquer le même raisonnement : builder localement les images concernées, et idéalement republier les images corrigées sur un registry maîtrisé.

7. Recommandations complémentaires

7.1 Segmentation réseau

PostgreSQL doit être isolé sur un réseau `dbnet` distinct :

Listing 7.1 – docker-compose.yml — segmentation réseau cible

```
1 networks:
2   appnet:
3     driver: bridge
4   dbnet:
5     driver: bridge
6
7 services:
8   postgres:
9     networks:
10      - dbnet          # isole du reste
11
12   ms-exercise:
13     networks:
14      - appnet
15      - dbnet          # acces autorise a la DB
16
17   gateway:
18     networks:
19      - appnet          # pas d'accès a dbnet
```

7.2 Cloisonnement des secrets

Chaque service ne doit recevoir que les variables strictement nécessaires. Concrètement, plutôt qu'un unique `env_file: variables.env` global, chaque service déclare son propre fichier d'environnement :

Listing 7.2 – docker-compose.yml — env_file par service

```
1 gateway:
```

```
2   env_file: ./env/gateway.env
3   # contient : SECRET_JWT, INTER_SERVICE_SECRET
4
5 ms-exercise:
6   env_file: ./env/ms-exercise.env
7   # contient : SECRET_JWT, INTER_SERVICE_SECRET, POSTGRES_*
8
9 ms-other:
10  env_file: ./env/ms-other.env
11  # contient : SECRET_JWT, INTER_SERVICE_SECRET, POSTGRES_*
12
13 postgres:
14  env_file: ./env/postgres.env
15  # contient : POSTGRES_PASSWORD uniquement
```

En production, l'utilisation de Docker Secrets ou d'un gestionnaire type HashiCorp Vault apporte des garanties supplémentaires — audit des accès, révocation à chaud, secrets non visibles dans `env` — sans pour autant constituer une protection absolue contre un attaquant ayant déjà un shell dans un container (voir section 3.2). La mesure la plus efficace reste la combinaison du cloisonnement et des tokens à courte durée de vie.

7.3 Validation du token inter-service côté microservices

Les microservices doivent vérifier explicitement le header `x-service-token` avant d'accepter toute requête, et rejeter avec un `401` tout appel ne provenant pas de la gateway :

Listing 7.3 – Middleware de vérification inter-service

```
1 function checkServiceToken(req, res, next) {
2   const token = req.headers['x-service-token'];
3   if (!token) return res.status(401).json({ error: 'Forbidden' });
4   try {
5     jwt.verify(token, process.env.INTER_SERVICE_SECRET, {
6       issuer: 'gateway',
7       audience: 'internal-service'
8     });
9     next();
10  } catch (e) {
11    return res.status(401).json({ error: 'Invalid service token' })
12      ;
13  }
14 }
```

Ce middleware doit être ajouté *avant* `auth.isAuth` dans chaque route exposée par `ms-exercise` et `ms-other`.

8. Conclusion

L'analyse du dépôt `SOY_MS_HTTP` (v5-http) a mis en évidence une architecture reposant entièrement sur le modèle Implicit Trust. La frontière réseau Docker `appnet` est la seule barrière de sécurité : dès qu'un container est compromis, l'attaquant dispose d'un accès complet à tous les services internes et à l'unique base de données partagée, sans aucune élévation de privilège supplémentaire.

Quatre failles critiques ont été identifiées — absence de token inter-service, secret JWT commun à tous les services, absence de vérification d'origine dans les deux microservices — auxquelles s'ajoute l'accessibilité directe de PostgreSQL depuis l'ensemble du réseau.

La correction prioritaire mise en place — token inter-service signé avec un secret dédié, injecté par la gateway avant chaque proxy — constitue le premier pas concret vers une architecture Zero Trust. Le build local de la gateway dans le `docker-compose.yml` est la conséquence directe et *uniquement pratique* de cette modification : l'image publiée sur le registry ne contenant pas notre code ajouté, il faut construire l'image localement ; cet argument ne vaut pas pour les autres services dont le code source n'a pas été touché.

Les recommandations complémentaires (segmentation réseau, cloisonnement des secrets par service, middleware de validation côté microservices) permettront de compléter cette transition. On notera que la version 7 de ShellOnYou va déjà dans ce sens en dotant chaque microservice de sa propre base de données isolée, réduisant ainsi mécaniquement la portée d'un mouvement latéral au seul périmètre du service compromis.

Bibliographie

- [1] MITRE ATT&CK, *Lateral Movement* (TA0008), <https://attack.mitre.org/tactics/TA0008/>, consulté en mai 2026.
- [2] NIST SP 800-207, *Zero Trust Architecture*, National Institute of Standards and Technology, 2020. <https://doi.org/10.6028/NIST.SP.800-207>
- [3] Docker Inc., *Manage sensitive data with Docker secrets*, <https://docs.docker.com/engine/swarm/secrets/>, consulté en mai 2026.